

Lab 3 Report

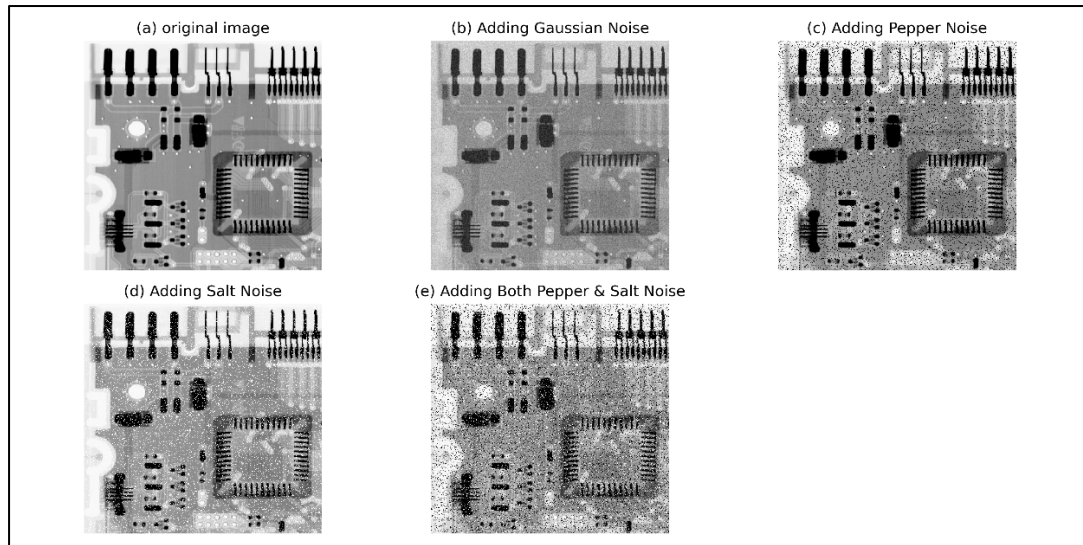
112062119 楊竺函

Proj05-01: Noise Generators

1. Implementation

- (1) 讀入 input.tif，並以 grayscale 形式讀取成 2-D image
- (2) 將輸入影像轉為 np.uint8 型態，並且 pixel value normalize 成 [0,255]
- (3) 使用 addGaussianNoise() 對原始影像加入 Gaussian noise
 - 使用 np.random.normal() 產生 Gaussian noise array
 - 將 Gaussian noise 直接加到原始影像上
 - 將結果 normalize 回 0~255，並轉回 np.uint8
- (4) 使用 addImpulseNoise() 對原始影像加入 impulse noise
 - 使用 np.random.rand(*input_s.shape) 產生與輸入影像相同大小、範圍為 0~1 的 random matrix
 - 若 random value 小於 P_p ，則將該 pixel 設為 0，代表 pepper noise
 - 若 random value 介於 P_p 與 $P_p + P_s$ 之間，則將該 pixel 設為 255，代表 salt noise
- (5) 產生只含 pepper noise、只含 salt noise，及包含 pepper + salt noise 的影像
- (6) 將各種 noisy image 輸出成影像檔，方便觀察不同 noise model 的效果
- (7) 將原始影像與各種 noisy image 彙整並利用 pyplot 畫出結果圖

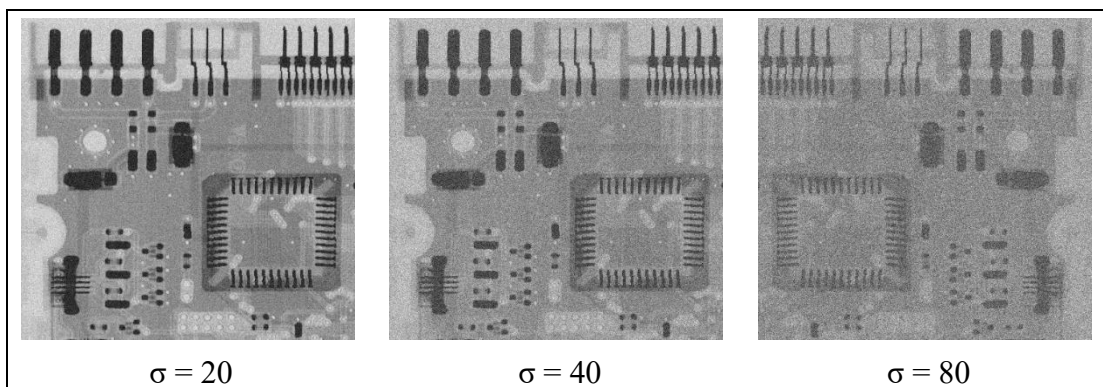
2. Execution Result



註：圖 (b) 中所用的參數為 $\sigma = 20, \mu = 0$

3. Discussion

Gaussian Noise with Different σ



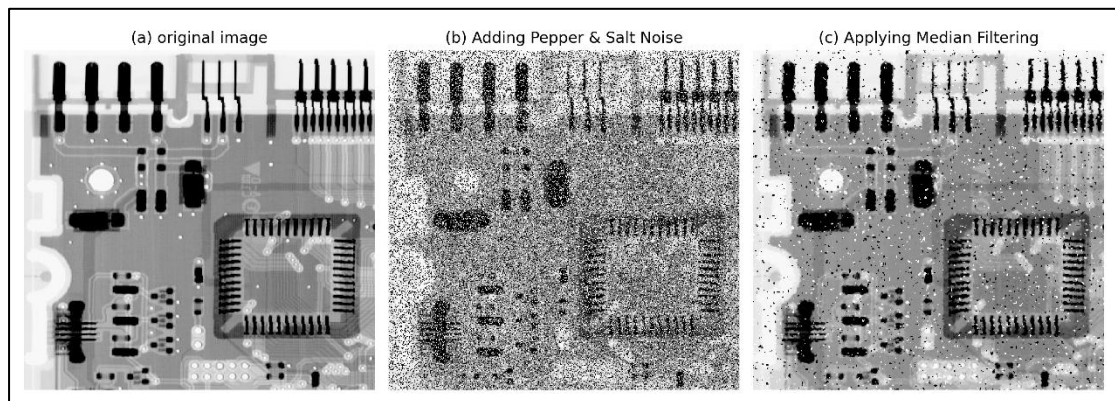
從不同 σ 值的 Gaussian noise 結果可以觀察到，當 σ 增加時，影像受到雜訊干擾的程度也會明顯提高。當 $\sigma = 20$ 時，影像中只出現較細微的灰階顆粒，原本電路板上的主要結構與邊緣仍然相當清楚；當 $\sigma = 40$ 時，雜訊變得更加明顯，平滑區域開始出現較多亮暗不均的顆粒，但大部分物件輪廓仍可辨識；當 $\sigma = 80$ 時，雜訊強度大幅增加，整張影像的灰階變化被明顯擾亂，部分細節與邊緣變得模糊，原始影像中的結構也較難清楚分辨。這是因為 σ 代表 Gaussian noise 的標準差， σ 越大表示隨機雜訊值偏離平均值的程度越高，因此加到影像後會造成更大的 pixel intensity 變化。由此可知，Gaussian noise 的 σ 值會直接影響影像品質，較大的 σ 會使影像產生更強烈的破壞。

Proj05-02: Noise Reduction Using a Median Filter

1. Implementation

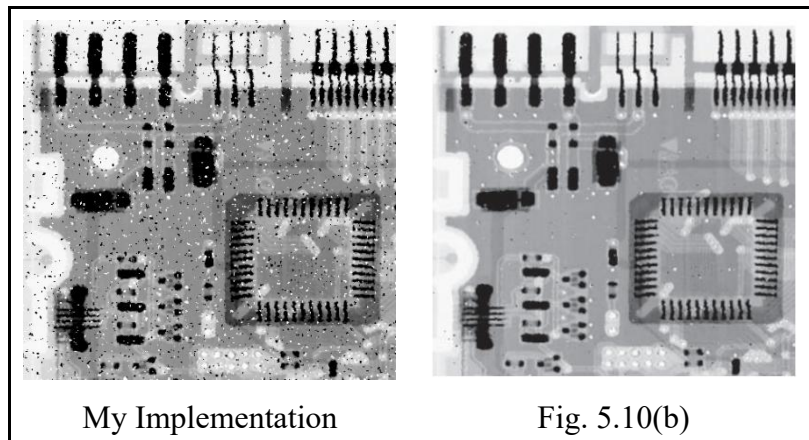
- (1) 讀入 input.tif
- (2) 使用 `normalize_to_uint8()` 將影像轉換為 `np.uint8`，並 `normalize` 到 `[0,255]`
- (3) 使用 Proj05-01 中的 `addImpulseNoise()` 對影像加入 salt-and-pepper noise
 - $P_s = 0.2$ 代表 salt noise 的機率， $P_p = 0.2$ 代表 pepper noise 的機率
 - 將加入雜訊後的影像輸出為 `out_pepper_and_salt.tif`
- (4) 使用 `myMedianFilter()` 對 noisy image 進行 median filtering
 - 在 `myMedianFilter()` 中，先使用 `reflect padding` 處理影像邊界
 - 對每個 pixel 取其周圍 `3 x 3` 的 neighborhood
 - 將 `3 x 3` window 中的 9 個 pixel value 排序
 - 取排序後的中位數作為輸出影像中對應位置的 pixel value
 - 將 median filtering 後的結果輸出為 `restored_img.tif`
- (5) 利用 `pyplot` 將 original image、noisy image 與 restored image 組合成圖

2. Execution Result



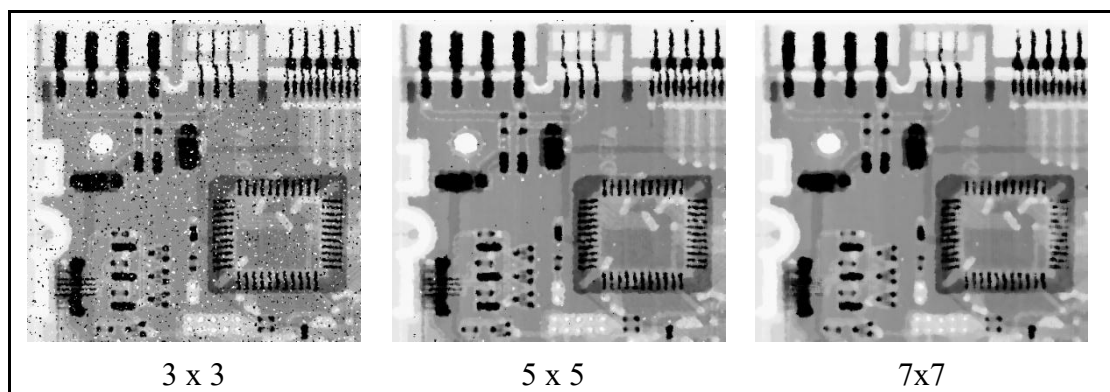
3. Discussion

3.1 Difference Between Execution Result & Fig. 5.10(b)



本實作結果與 Fig. 5.10(b) 的主要差異在於殘留雜訊較多、影像也較模糊。原因是 Fig. 5.10(a) 使用的 salt-and-pepper noise 機率為 $P_s = P_p = 0.1$ ，而本作業設定為 $P_s = P_p = 0.2$ ，代表受到污染的像素比例約為課本範例的兩倍。當雜訊比例變高時， 3×3 median filter 的鄰域中可能有超過半數像素已被污染，因此中位數仍可能是 0 或 255，導致部分雜訊無法被移除。因此，本實作雖然能明顯降低 salt-and-pepper noise，但結果比 Fig. 5.10(b) 保留更多黑白雜點，且細節與邊緣也會因 median filtering 而略為平滑。

3.2 Different Filter Size



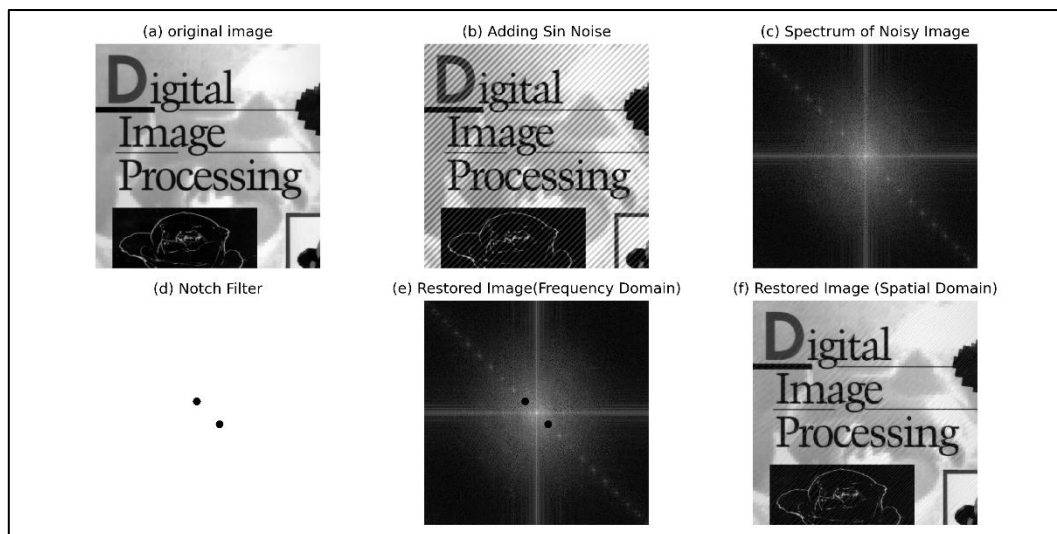
使用 3×3 median filter 時，影像中的細節與邊緣保留較多，但由於加入的雜訊比例較高，仍可觀察到許多殘留的黑白雜點。當 filter size 增加為 5×5 時，median filter 會參考更大的鄰域，因此能更有效抑制 0 與 255 雜訊，影像看起來明顯較乾淨。但隨著 filter size 增大，輸出像素會受到更大範圍鄰域影響，細線與元件邊緣也會被平滑。當 filter size 增加到 7×7 時，雖然雜訊殘留最少，但影像中的電路細節、邊緣與小型結構也變得更模糊。因此，median filter size 的選擇反映了去雜訊能力與細節保留之間的取捨。

Proj05-03: Periodic Noise Reduction Using a Notch Filter

1. Implementation

- (1) 讀入 `input.tif`，將影像轉成 `float32` 型態，並 `normalize` 到 `range 0~1`
- (2) 使用 `addSinNoise()` 在 `spatial domain` 中加入 `sinusoidal noise`
- (3) 將 `noisy image` 乘上 $(-1)^{(x+y)}$ ，使轉換後的 `spectrum` 中心移到影像中央
- (4) 使用 `numpy.fft.fft2()` 將 `noisy image` 轉換到 `frequency domain`
- (5) 計算頻譜 `magnitude`，並使用 $\log(1 + |F(u,v)|)$ 增強 `spectrum` 的顯示效果
- (6) 使用 `notchFiltering(input_f, D0, u0, v0)` 產生 `ideal notch reject filter`
 - `Notch filter` 會在 `centered frequency domain` 中對應到 $(+u_0, +v_0)$ 與 $(-u_0, -v_0)$ 的位置挖掉兩個圓形區域
 - 在 `frequency domain` 中將 `noisy spectrum` 與 `Notch filter` 相乘，以移除週期性雜訊對應的頻率成分
- (7) 使用 `numpy.fft.ifft2()` 將濾波後的結果轉回 `spatial domain`
- (8) 再乘一次 $(-1)^{(x+y)}$ ，將影像移回原本的 `spatial domain` 位置
- (9) 將還原後影像 `clip` 到 `range 0~1`，並轉回 `float32` 型態
- (10) 使用 `computePSNR()` 計算原圖與 `restored image` 之間的 `PSNR`
- (11) 將各階段結果輸出成影像檔，並利用 `pyplot` 畫出包含原圖、`noisy image`、`frequency spectrum`、`Notch filter`、`filtered spectrum` 與 `restored image` 的結果圖

2. Execution Result

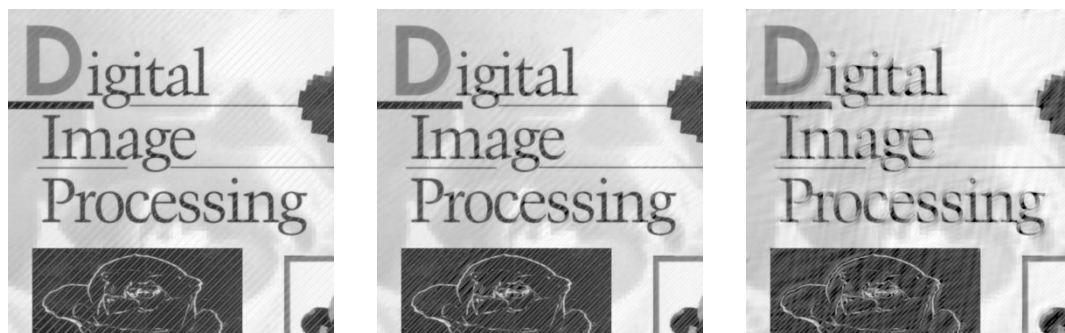


[OUTPUT] PSNR between input and restored image = 30.522666931152344 dB

使用的參數: $A = 40/255$, $u_0 = v_0 = 35$, $D_0 = 12$

3. Discussion

Different D_0 Values



$D_0 = 2$, PSNR = 31.11 dB

$D_0 = 12$, PSNR = 30.52 dB

$D_0 = 30$, PSNR = 25.26 dB

D_0 代表在 frequency domain 中被移除的半徑範圍，影響週期性雜訊的去除程度與影像細節的保留。當 $D_0 = 2$ 時，filter 移除的頻率範圍過小，只能去除雜訊頻率中心附近很少的成分，因此修復影像中可看到明顯的斜向條紋殘留。不過由於移除的正常影像資訊較少，PSNR 仍可達到 31.11 dB。當 $D_0 = 12$ 時，notch filter 能較完整地涵蓋 sinusoidal noise 在頻譜中造成的亮點，使週期性條紋明顯減弱，同時保留大部分影像內容與文字邊緣，因此得到 30.52 dB 的 PSNR，視覺上較為平衡。當 $D_0 = 30$ 時，雖然濾除範圍更大，但也同時移除原始影像中接近該頻率位置的正常成分，使影像細節受到明顯影響，因此 PSNR 下降至 25.26 dB。因此， D_0 並非越大越好，而是需要在雜訊移除效果與影像細節保留間取得平衡。

Proj05-04: Parametric Wiener Filter

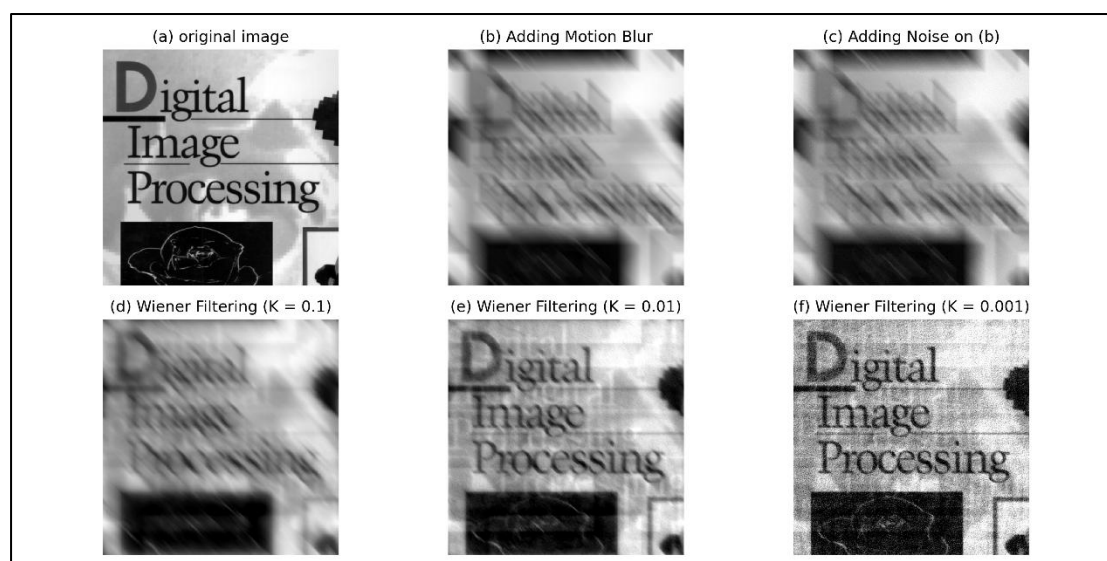
1. Implementation

- (1) 讀入 input.tif，並轉為 float32 型態
- (2) 使用 to_frequency_domain() 將影像轉換至 frequency domain
 - 使用 shiftSpectrum() 將影像乘上 $(-1)^{(x+y)}$ ，使頻譜中心移至影像中央
 - 使用 numpy.fft.fft2() 將影像轉換至 frequency domain
- (3) 使用 addMotionBlur() 產生 motion blur degradation function $H(u,v)$ 及用 H 對影像進行破壞
 - 根據 Eq. 5-77 建構 motion blur degradation function 並處理 zero division 問題
 - 在 frequency domain 中將輸入影像頻譜與 $H(u,v)$ 相乘，得到 motion blurred image 的頻譜
- (4) 使用 to_spatial_domain() 將影像轉回 spatial domain
 - 使用 numpy.fft.ifft2() 將 motion blurred image 轉回 spatial domain
 - 使用 shiftSpectrum() 將影像乘上 $(-1)^{(x+y)}$ ，使影像移回原處
- (5) 使用 addGaussianNoise() 對 blurred image 加入 $\mu=0$ ， $\sigma=\sqrt{10}$ 的 Gaussian noise
- (6) 再次將 blurred noisy image 轉換至 frequency domain
- (7) 使用 wienerFiltering(input_f, H, K) 進行 Wiener filtering
 - 根據 Eq. 5-85 分別使用 $K=0.1$ 、 $K=0.01$ 、 $K=0.001$ 對影像進行還原
 - 將每個 restored image 轉回 spatial domain
 - 使用 computePSNR() 計算原圖與不同 K 值復原結果之間的 PSNR
- (8) 將原圖、motion blurred image、blurred noisy image，以及三種 K 的 Wiener filtering 結果彙整成 result.png

註 1: shiftspectrum() 為 lab3 實作之函式，此處沿用原程式碼故不贅述

註 2: computePSNR() 為 Proj05-03 中實作之 function，addGaussianNoise() 則為 Proj05-01 中所實作之 function

2. Execution Result



K	0.1	0.01	0.001
PSNR	15.72 dB	18.94 dB	17.13 dB

使用的參數: $T = 1$, $a = b = 0.1$, $\mu = 0$, $\sigma = \sqrt{10}$, $K = 0.1 / 0.01 / 0.001$

3. Discussion

3.1 比較討論不同 K 得到的視覺效果和 PSNR

在本實驗中，我分別使用 $K = 0.1$ 、 $K = 0.01$ 與 $K = 0.001$ 進行 Wiener filtering，並比較不同 K 值對影像還原結果的影響。由結果可以觀察到， K 值會影響濾波器在去除 motion blur 與抑制 Gaussian noise 之間的取捨。當 $K = 0.1$ 時，濾波器較保守，對雜訊的放大程度較低，但也使得對模糊的修復能力較弱，影像中的文字與邊緣仍不夠清楚，PSNR 為 15.72 dB。當 K 降低至 0.01 時，影像的文字輪廓變得較明顯，同時背景雜訊仍維持在可接受範圍內，因此在視覺上取得較好的平衡，PSNR 也提升至 18.94 dB，為三組參數中最高的結果。

當 K 進一步降低至 0.001 時，濾波器更接近 inverse filtering，對模糊的還原效果最強，文字看起來最清楚。然而，過小的 K 會使濾波器在退化函數 $H(u,v)$ 較小的頻率位置產生較大的增益，因此 Gaussian noise 也被明顯放大，使影像出現較強的顆粒感與雜訊。雖然 $K = 0.001$ 在視覺上具有較高的銳利度，但整體像素誤差增加，因此 PSNR 下降至 17.13 dB。

綜合來看，若以 PSNR 作為評估標準， $K = 0.01$ 在本實驗中提供了較佳的還原結果。它不像 $K = 0.1$ 那樣過於模糊，也不像 $K = 0.001$ 那樣放大過多雜訊，因

此能在去模糊與抑制雜訊之間取得較好的平衡。不過，若單純從人眼觀察的視覺清晰度來看， $K = 0.001$ 的文字與邊緣較為明顯，可能是因為人眼對局部文字結構與邊緣清晰度較敏感，而對部分隨機雜訊的容忍度相對較高。因此，PSNR 較高的結果不一定完全等同於主觀視覺上最清楚的結果。

3.2 Artifact Occurrence When $K = 0.001$

從還原結果中也可以觀察到，當 K 較小時，影像中除了雜訊被放大外，也可能出現較明顯的條紋或不自然紋理。這是因為頻率域復原會增強部分原本被 motion blur 衰減的頻率成分，但這些頻率中同時包含雜訊與數值誤差，因此在空間域中可能形成 artifacts。這也說明影像復原並非單純追求越強的反向補償越好，而需要在細節恢復與 artifact 抑制之間取得平衡。